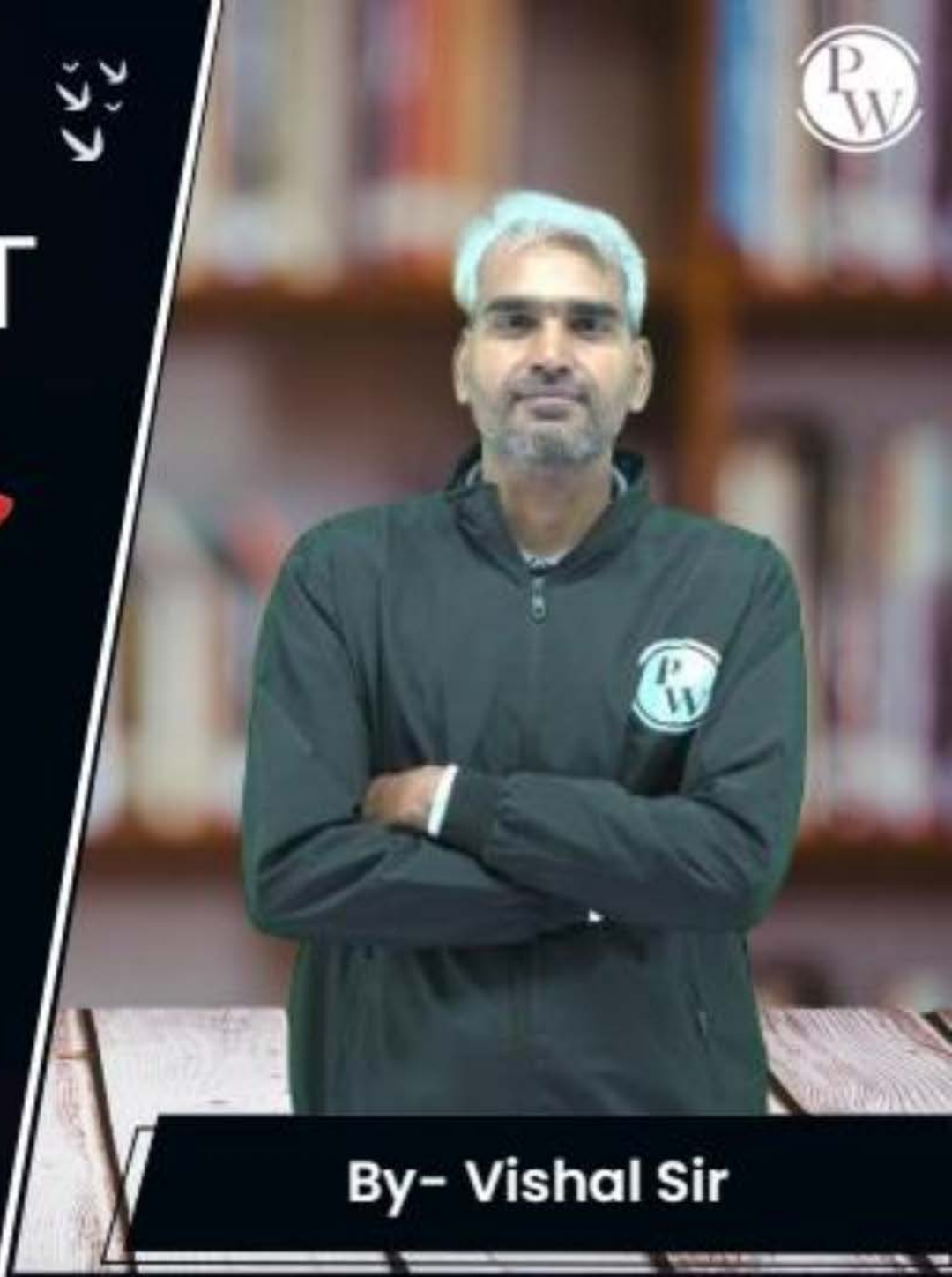
Computer Science & IT



Compiler Design

Syntax Analysis (Parser)

Lecture No. 10



By- Vishal Sir

Recap of Previous Lecture



Topic LR(0) Parsing

Slide

Topics to be Covered



Topic SLR(1) Parser

Topic CLR(1) & LALR(1) Parser

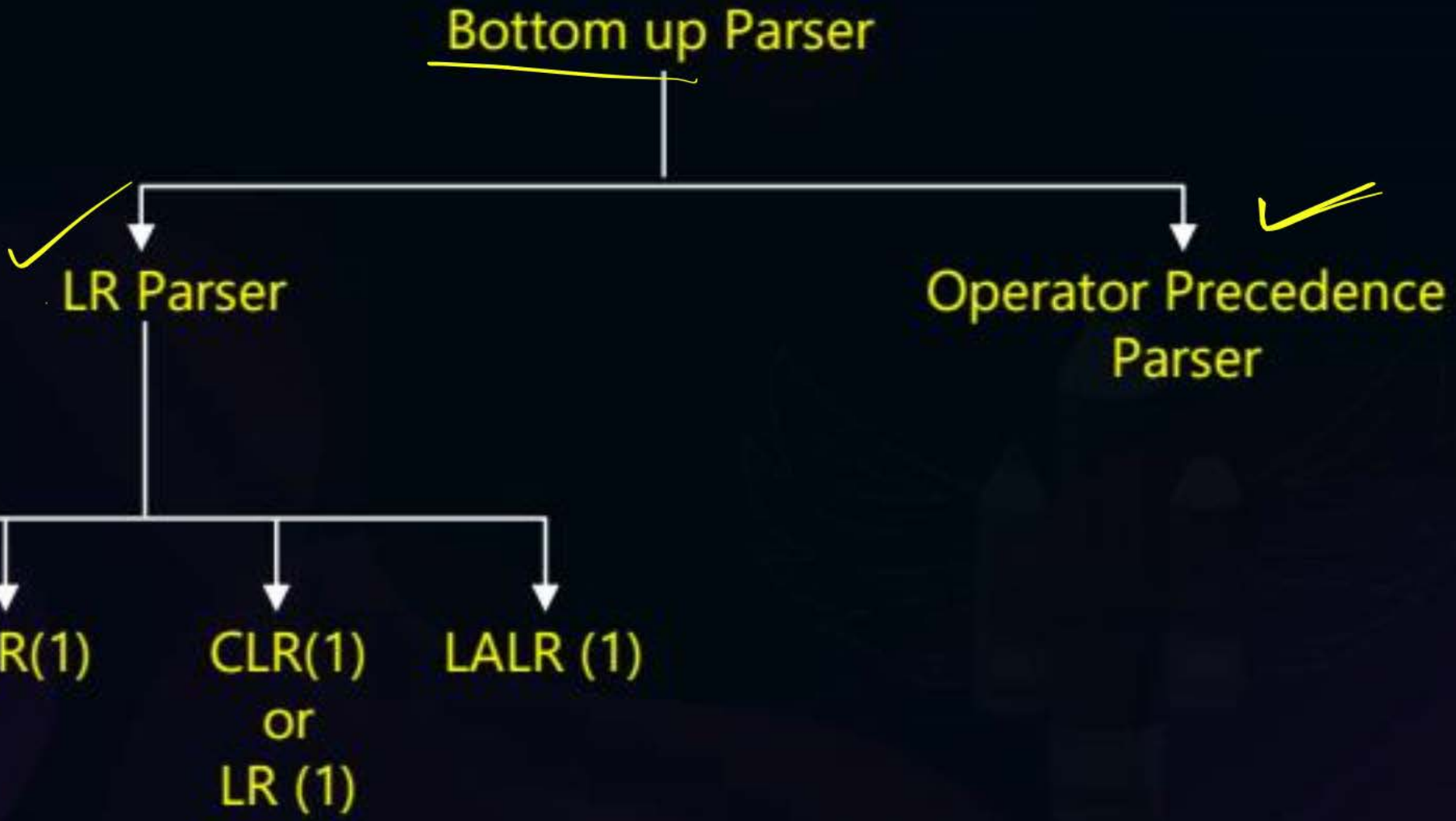


Topic : Bottom-up Parsers

- Bottom Up Parser starts constructing the parse tree starting from the given input string and proceeds until the starting symbol of the grammar is obtained.
- If a parser uses bottom up approach for parsing the string, then it follows reverse of rightmost derivation in the construction of parse tree.



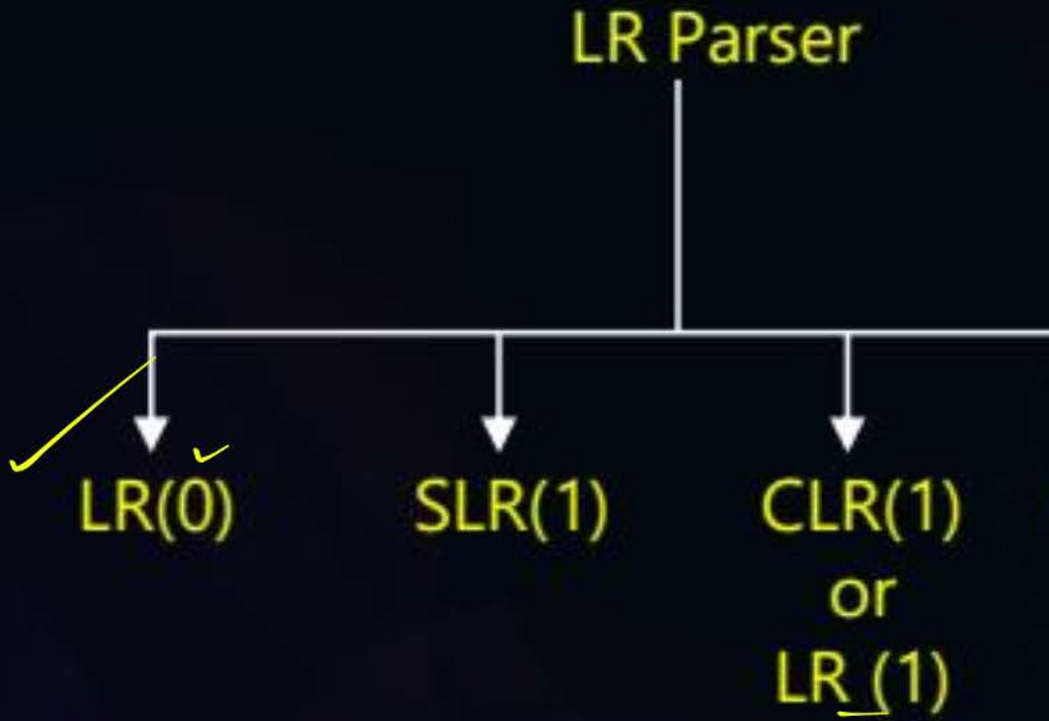
Topic : Types of Bottom-up Parsers



Slide



Topic : Types of LR Parsers



Slide



Topic : Handle

Handle is a sub-string in the right sentential form {i.e. sentential form w.r.t. right most derivation} that matches with the Complete right hand side part of a production of the grammar and Can be reduced into the non-terminal On the left hand side of that production.

Slide

Topic : Handle

Consider the following grammar

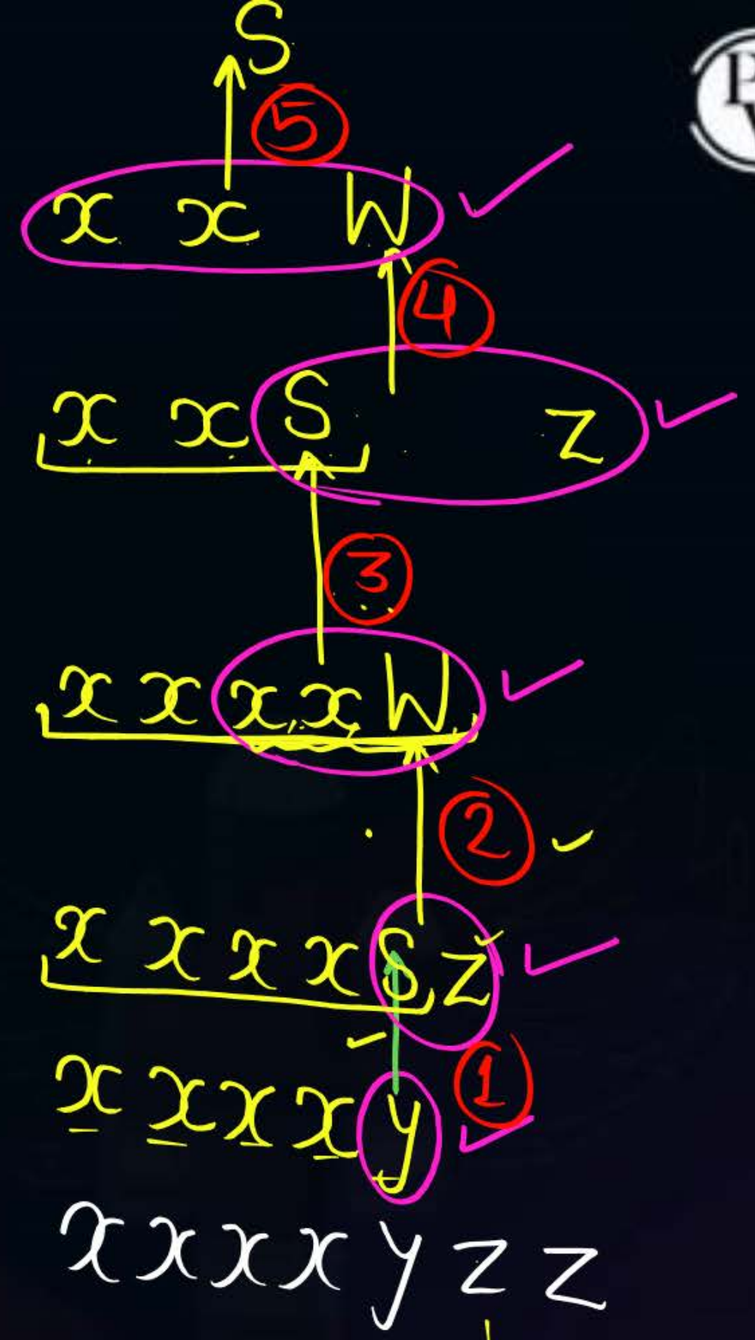
$S \rightarrow xxW$

$S \rightarrow y$

$W \rightarrow Sz$

How many numbers of handles are detected by bottom-up parser for the string = x x x x y z z





Slide

* Number of handles detected by bottom-up parser = Number of Reduce actions Performed by bottom-up Parser



Topic : Augmented Grammar

Augmented grammar is the same grammar With an extra production {known as augmented production} of type $S' \longrightarrow S$

Augmented production

Where,
S is the starting symbol of original grammar,
and S' is a newly added non-terminal

Slide

* Augmented production is added into the grammar to represent Successful parsing.

Whenever we reduce S into S' it represents Successful parsing

eg:

Grammar

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$



Augmented grammar

$S' \rightarrow S]$ ← Augmented Production

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$



Topic : LR(0) item

* LR(0) item is a production of the grammar having a DOT (.) mark somewhere on the RHS part of the production

eg: $A \rightarrow \alpha . \beta$ { α, β may be any thing }

Slide



$A \rightarrow XYZ$ is the Production

Topic : LR(0) item

eg i) $A \rightarrow .XYZ$ { We have seen nothing, and we are ready to see the string derivable from XYZ }

ii) $A \rightarrow X.YZ$ { We have seen the string derivable from X, And we are ready to see the string derivable from YZ }

iii) $A \rightarrow XY.Z$ { We have seen the string derivable from XY, And we are ready to see the string derivable from Z }

iv) $A \rightarrow XYZ.$ { We have seen the string derivable from XYZ. And there is nothing to be seen i.e. We are ready to reduce w.r.t $A \rightarrow XYZ$ }

Slide

$A \longrightarrow .\epsilon \equiv A \longrightarrow \epsilon.$

We have seen nothing and, there is nothing to see. $\equiv$ We have seen nothing, and there is nothing to see

If we have $A \longrightarrow .\epsilon$ then we can simply convert it into $A \longrightarrow \epsilon.$ {i.e. we are ready to reduce w.r.t $A \longrightarrow \epsilon$}

Slide

Closure of LR(0) item :-

→ Obtain closure of LR(0) item I { i.e. Closure(I) }

→ Step-1 :- Include Item I itself into closure(I)

→ Step2 :- If there exists any item of the form $A \rightarrow \alpha.B\beta$ in closure(I) such that $B \rightarrow \gamma$ is a production in the grammar and $B \rightarrow .\gamma$ is not already in closure(I), then add $B \rightarrow .\gamma$ in closure(I).

{ $\alpha, \beta \in (V+T)^*$ and B is a non-terminal

Slide

↳ Step 3:- Repeat step-2 for every newly added Production (if possible)

eg:-

Grammar

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

$\Rightarrow$ Augmented grammar

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA$

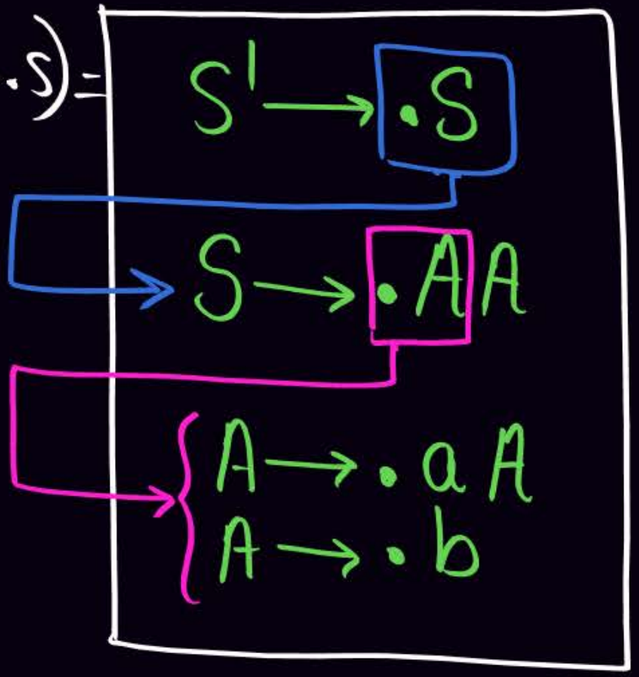
$A \rightarrow b$

* Consider the following LR(0) item

$S' \rightarrow .S$ ✓

obtain Closure $(S' \rightarrow .S)$

Closure $(S' \rightarrow .S) =$



* Construction of DFA of LR(0) items w.r.t. given grammar

* ① → obtain the augmented grammar,
  let, $S' \rightarrow S$ is the augmented production

② Construct States of the DFA :-

① Construct initial state

② Start a New State

③ Complete the Construction of newly started State

④ Repeat step② & step③ until no more state Can be started

① Construction of initial state of DFA of LR(0) items.

If $S' \rightarrow S$ is the augmented production, then Closure($S' \rightarrow .S$) will define the items in the initial state of the DFA of LR(0) items

② Starting a new state in the DFA of LR(0) items

* If $A \longrightarrow \alpha . X \beta$ is an item in the state 'i' of the DFA of LR(0) items, { where $\alpha, \beta \in (V+T)^*$ and X is a symbol of length one { terminal / Non-terminal } }

then locate all the items in state 'i' such that, X immediately follows the Dot(.) mark, and start a new state 'j' by moving the Dot(.) mark after X in all those items

③ Complete the Construction of newly started state:-

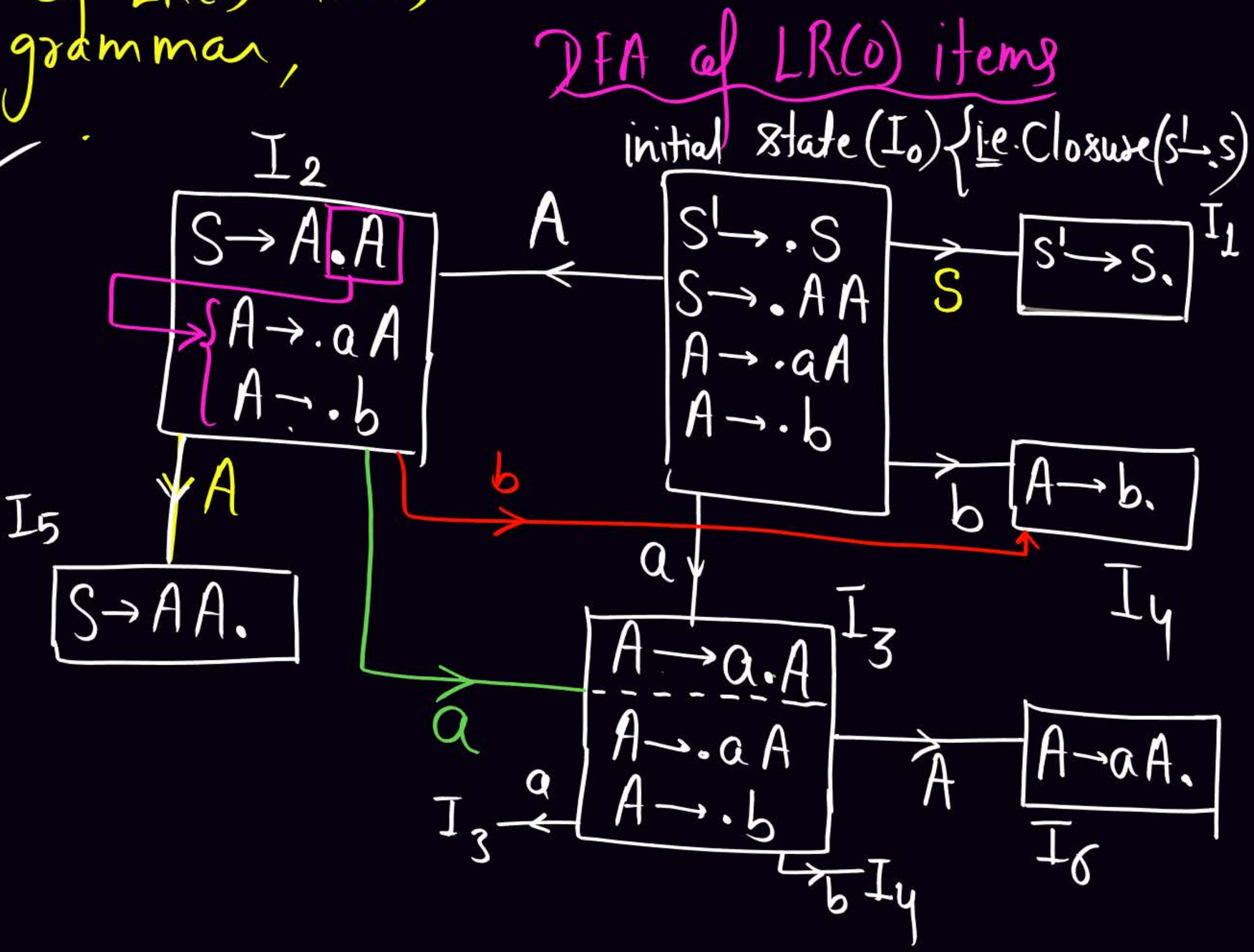
i.e obtain the closure of the LR(0) items with which the state is started.

* In newly started state 'j' if there exists an item of the form $A \longrightarrow \alpha.B\beta$ and $B \longrightarrow \gamma$ is a production of the gramman and if $B \longrightarrow .\gamma$ is not already present in state 'j' then add $B \longrightarrow .\gamma$ in state 'j'.

And Repeat this process for every newly added item in state 'j'.

④ Repeat ② & ③ until no more states can be started

* Construct the DFA of LR(0) items for the following grammar,



LR(0) Parser :-

① obtain augmented grammar

② Obtain DFA of LR(0) items

③ Convert DFA of LR(0) items into LR(0) Parsing table

④ If no entry of the Action part of LR(0) parsing table Contains more than one option then Corresponding grammar is LR(0) grammar

⑤ If any entry of the action part contain more than one option then not LR(0) grammar

✡ Conversion of DFA of LR(0) item into LR(0) Parsing table.

Note:- LR parsing table w.r.t. all LR parsers Consists of two parts
① Action Part ② Goto Part

→ Action Part of LR Parsing table will Consists of 4 types of entries

1. Shift
2. Reduce
3. Accept
4. Error

* Action part entries w.r.t. LR(0) parsing table

① **Shift** :- If state 'i' of the DFA of LR(0) items contains an item of the form

$A \longrightarrow \alpha . a \beta$ { where 'a' is any terminal } (must be terminal for shift)

and there is a transition out of state 'i' to state 'j' on terminal 'a', then

Update Action [i, a] = S_j { from state 'i' on terminal 'a' shift to state 'j' }

(i : state, a : terminal)

② Reduce :-

∴ If state 'i' of the DFA Contains an item of the form

$$A \longrightarrow \alpha .$$ { i.e ready to reduce w.r.t $A \longrightarrow \alpha$ }

Where $A \longrightarrow \alpha$ is the jth production of the grammar, then update R_j { i.e. Reduce w.r.t. Production 'j' } in all the entries of Action part Corresponding to State 'i'

Note { No look-ahead is used to place reduce entry in the parsing table } LR(0)

③ Accept :- If state 'i' of the DFA Contains an item of the form

$S' \longrightarrow S.$, {where $S' \longrightarrow S$ is augmented Production}

then update

Action [i, $] = Accept

④ Error :-

Set all other entries of action part as Error {i.e. blank entries in Action Part correspond to Error}

* Goto Part of LR parsing table :-

* Generally Goto part is same for all parsing table.

* Goto entries with respect to LR(0) Parser

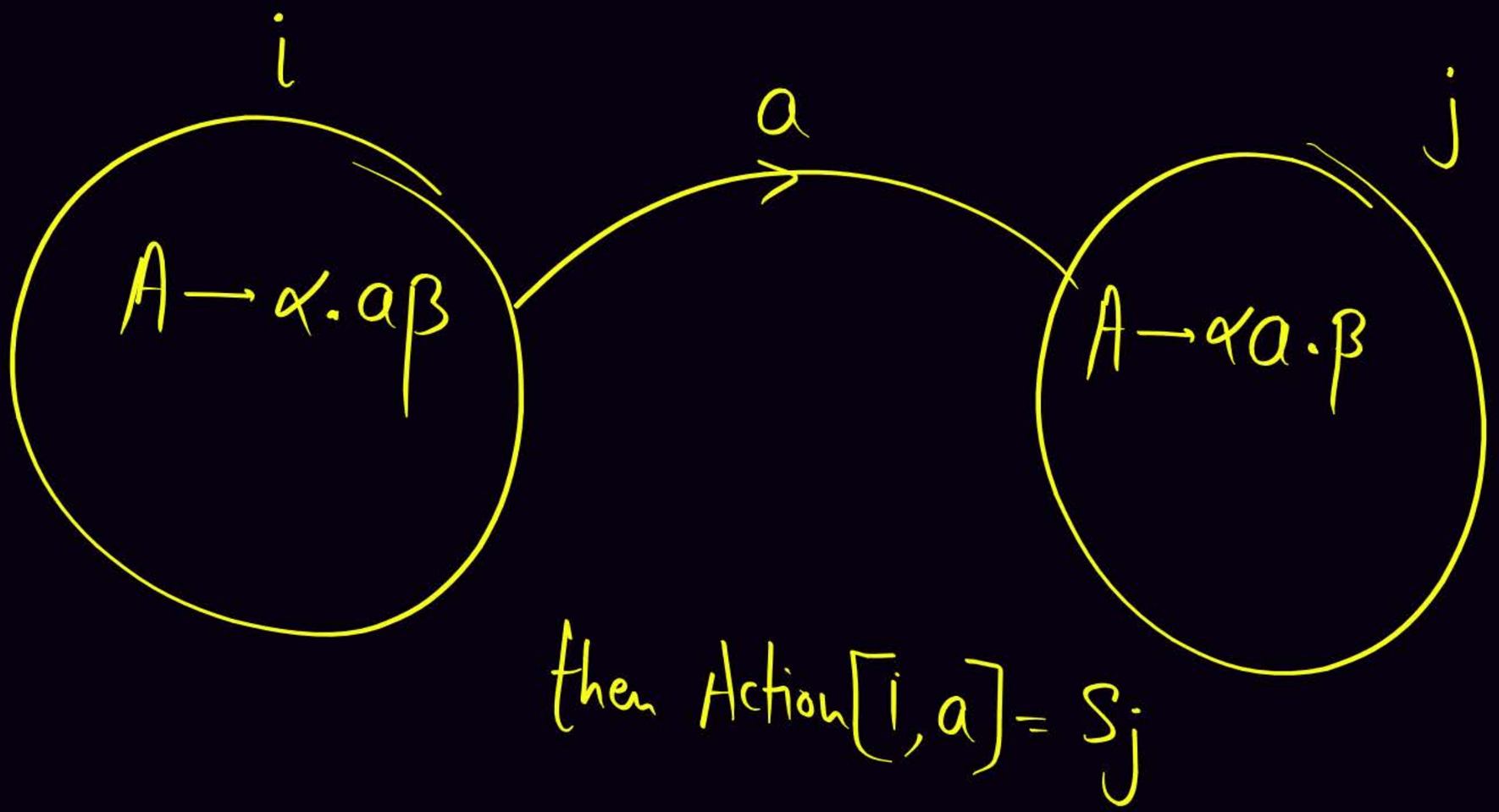
* If these exists any item in state 'i' of the form

$$A \longrightarrow \alpha . B \beta$$ (must be non-terminal) {where B is the non-terminal}

and these is a transition out of state 'i' to state 'j' on non-terminal B, then

update Goto [i, B] = j ← New State

State ← i, Non-terminal ← B



then Action[i, a] = s_j

* Construct LR(0) Parsing table for gramma

Production Number

① $S \rightarrow AA$

② $A \rightarrow aA$

③ $A \rightarrow b$

} DFA already known

LR(0) Parsing table for above gramma

	Action			Goto	
	a	b	$	S	A
0	S_3	S_4		1	2
1	Error	Error	Accept		
2	S_3	S_4			5
3	S_3	S_4			6
4	R_3	R_3	R_3		
5	R_1	R_1	R_1		
6	R_2	R_2	R_2		

States of the DFA

{ No entry of action part Contains more than one option

∴ above gramma is LR(0) gramma

Note:- Not every gramman is LR(0) gramman,

- There may be conflict in Action part of LR(0) parsing table
- If any entry of LR(0) parsing table Contains more than one option then it is called Conflict, If Conflict exists in LR(0) Parsing table then Corresponding gramman is not LR(0) gramman

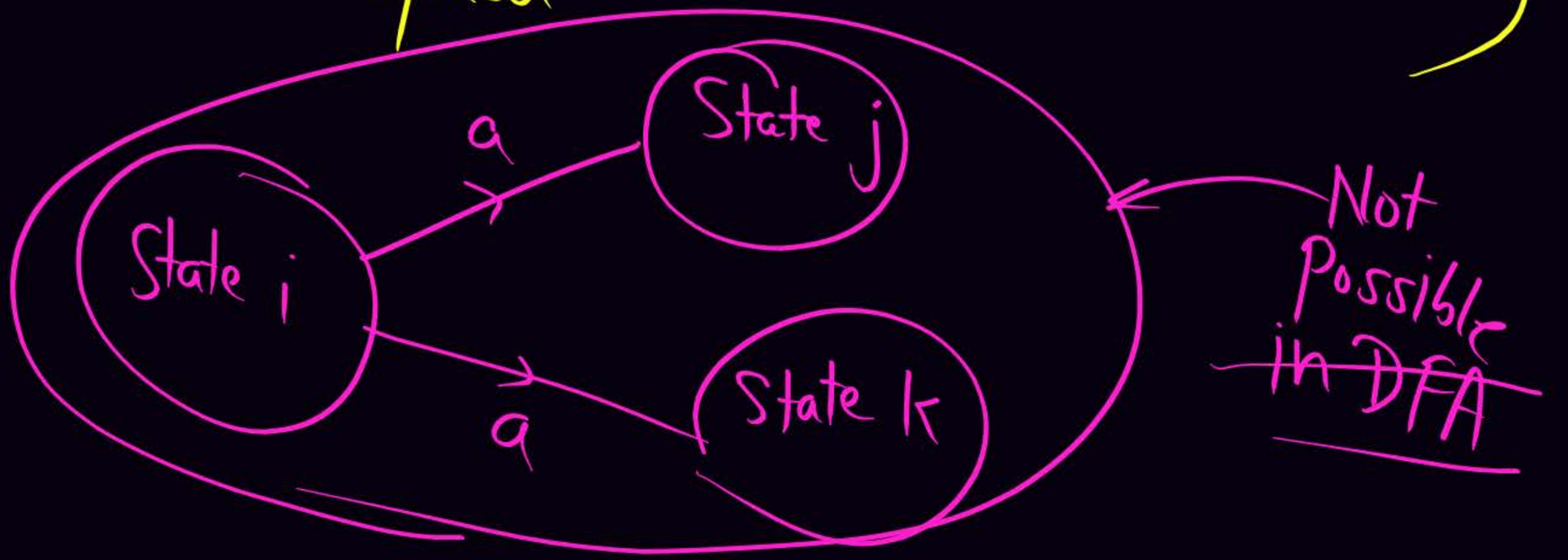
Types of Conflict in LR(0) Parsing table :-

① Shift-Reduce Conflict (SR Conflict) :- If shift and reduce actions are present in the same entry of LR(0) parsing table, then it corresponds to SR Conflict

② Reduce-Reduce Conflict (RR Conflict) :- If two or more reduce actions are present in the same entry of LR(0) parsing table, then it corresponds to RR Conflict

Note:- Shift-Shift Conflict is not possible because it is a DFA,

{ from one state we can not go to two different states on visiting the same symbol }

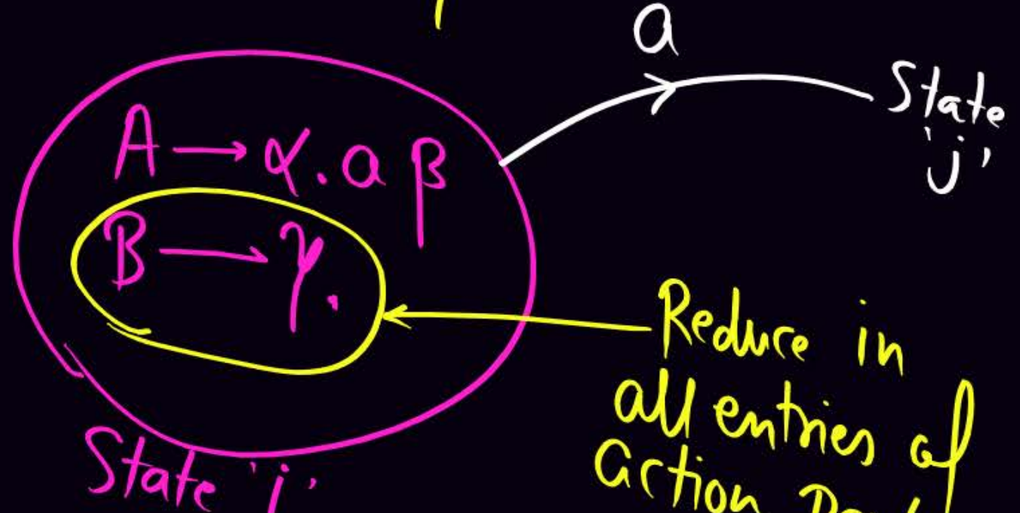


SR Conflict in LR(0) :-

If state 'i' of the dfa of LR(0) items contains items of the form, $A \rightarrow \alpha . a\beta$ (a: terminal)

$B \rightarrow \gamma .$

i.e.



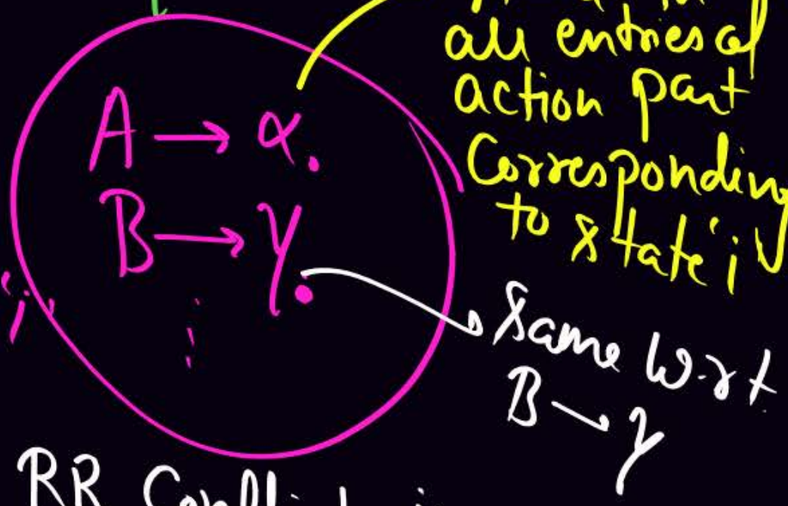
then SR Conflict exists in state 'i'

RR Conflict in LR(0) :-

If state 'i' of the dfa of LR(0) items contains items of the form

$A \rightarrow \alpha .$

& $B \rightarrow \gamma .$



then RR Conflict in state 'i'

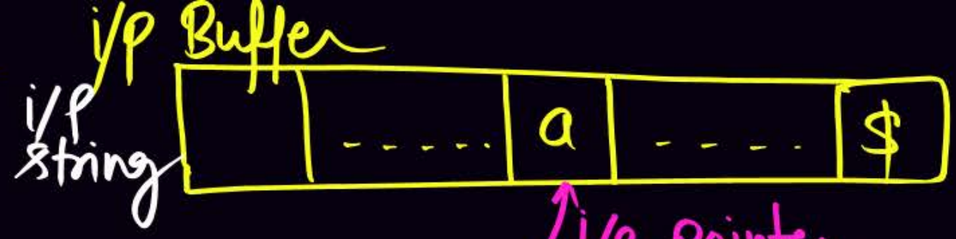
Note:- If there exists any state with SR or RR Conflict, then it is called **inadequate** state of the DFA.

* If there exists any inadequate state w.r.t. LR(0), then corresponding gramma is not LR(0) gramma.

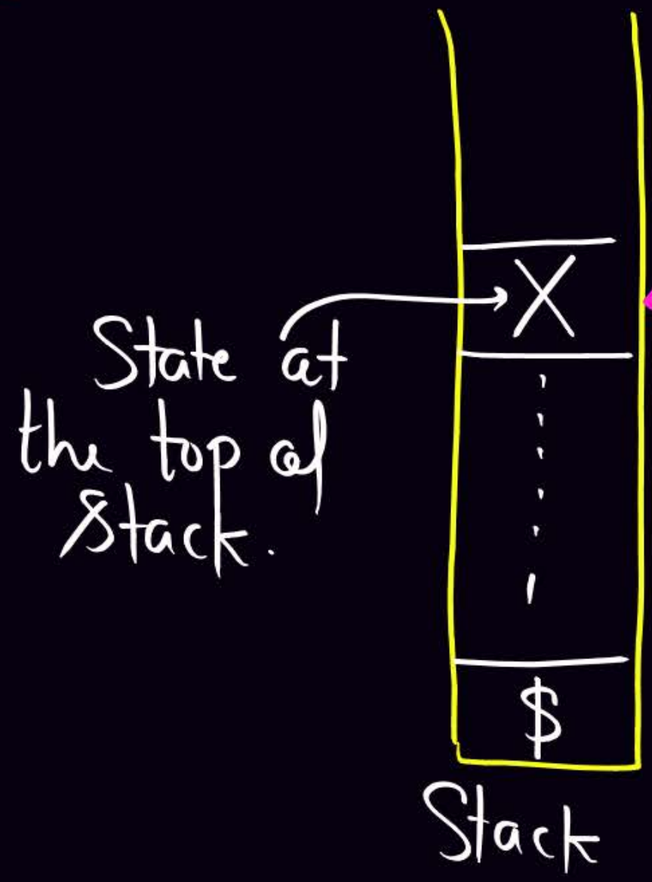
LR Parsing Algorithm :-

Note :- LR parsing algorithm will remain same for all types of LR parsers, but LR parsing table may differ as LR parser changes

LR Parsing Algorithm:-



i/p pointer (look-ahead symbol)



LR parser

LR parsing table

} May be different for different LR parsers

LR Parsing Algo:-

Initially PUSH $ into the stack, and then PUSH initial state of the DFA at top of stack

- Let X is the state at top of stack, and 'a' is the look-ahead symbol {i.e terminal pointed by input pointer}, then LR parser takes decision regarding shift or reduce action based on LR parsing table entry Action[X, a]

LR Parsing Algo :-

* Following are the possible actions based on LR parsing table entry.

① Shift :- If Action[X,a] = S_j, then shift (PUSH) 'a' into the stack and then shift state 'j' at the top of the stack, and finally increment the input pointer

LR Parsing Algo :-

② **Reduce :-** If Action [X, a] = R_j, {i.e. Reduce w.r.t j^{th} production}

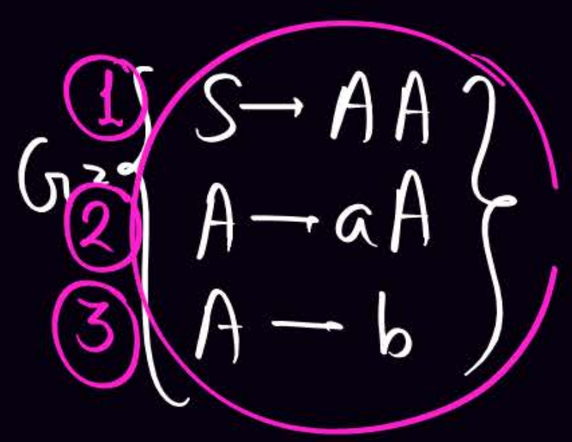
and if j^{th} production of the grammar is "$A \longrightarrow \beta$", then POP $2*|\beta|$ elements from the Stack and PUSH 'A' into the Stack, and then PUSH Goto [i, A] into Stack where 'i' is the state in the Stack before the non-terminal we just Pushed into the Stack (i.e A)

LR Parsing Algo:-

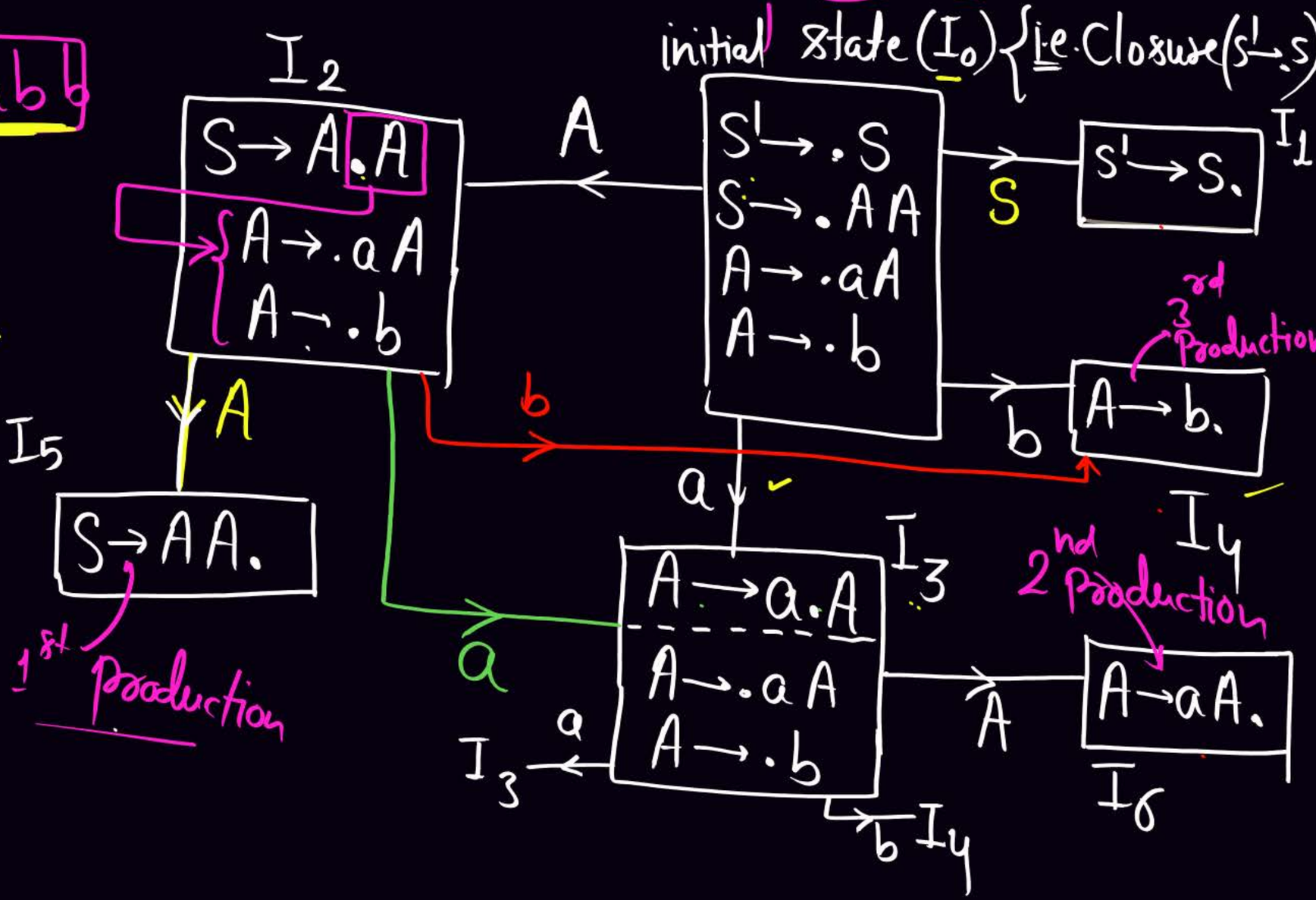
③ Accept:- If Action [X, a] = Accept, then Parsing is successful.

④ Error: If Action [X, a] = blank/Error, then Parser return Syntax Error.

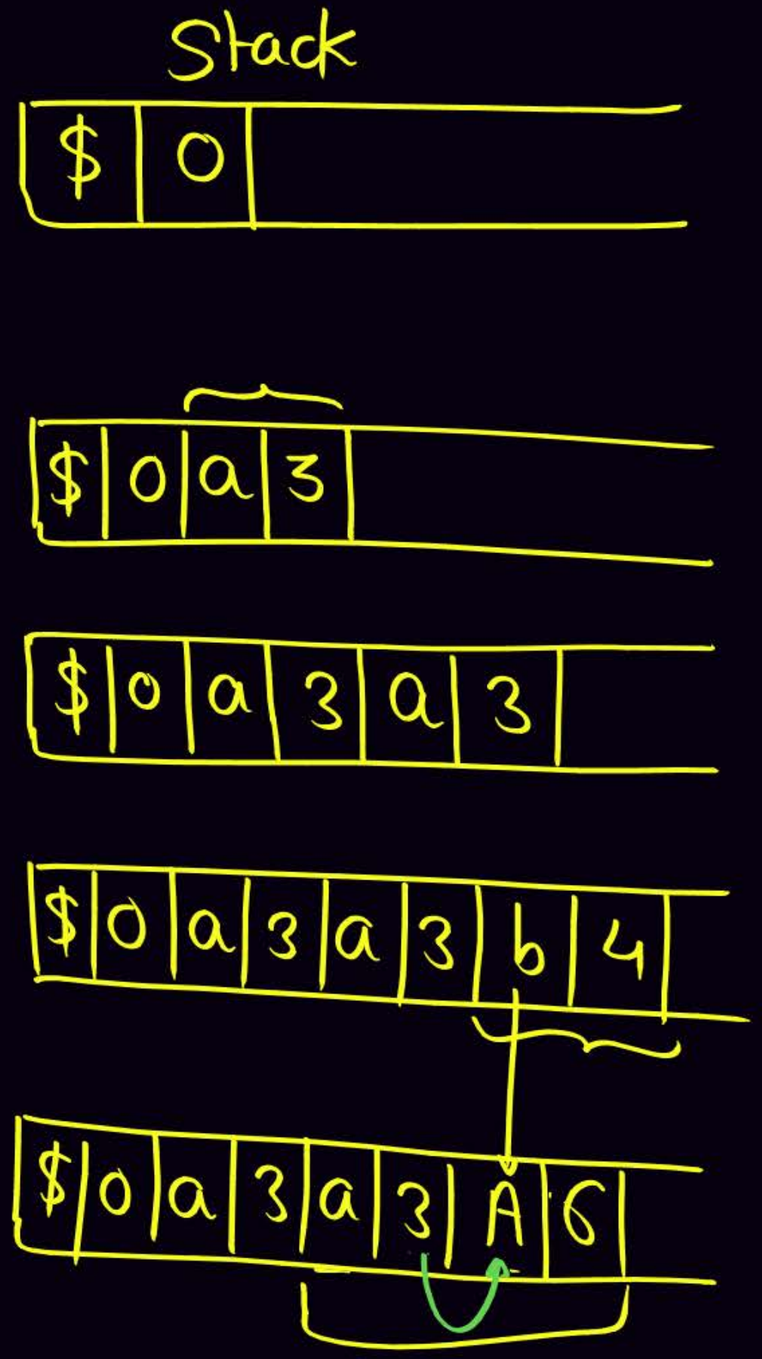
* Consider the following grammar & following LR(0) Parsing table



aabb



Q: Parse the string w = aabb $ using LR parsing Algo



Action[0, a] = S_3
{shift a,
shift 3, and
increment pointer

Action[3, a] = S_3

Action[3, b] = S_4

Action[4, b] = R_3
(Reduce w.r.t 3rd Production)
Pop 2*|b| = 2*1 elements, PUSH A,
PUSH(Goto[3, A] = 6)

a a b b $
↑

a b b $
↑

b b $
↑

b $
↑

b $
↑

Action [6, b] = R_2 ✓

2nd Production is $A \rightarrow aA$

∴ Pop $2 * |aA| = 2 * 2 = 4$ elements and PUSH A and PUSH Goto[3, A] = 6

a	3	A	6

Popped

$	0	a	3	A	6	

Action[6, b] = R_2 ✓

Same and PUSH Goto [0, A] = 2

$	0	A	2	

Action [2, b] = S_4

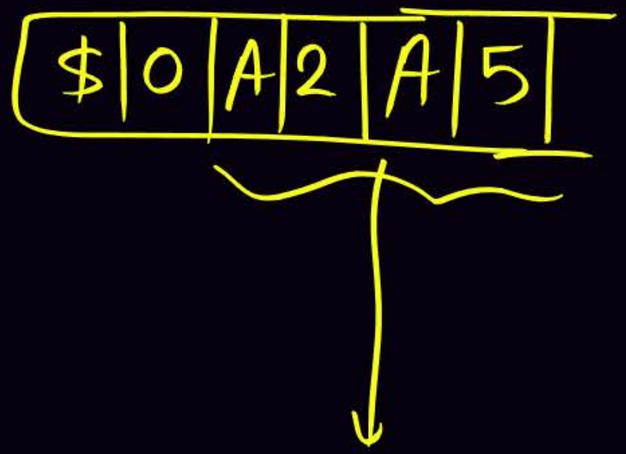
$	0	A	2	b	4

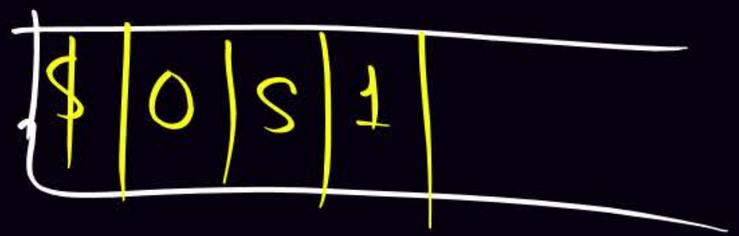
Action [4, $] = R_3

Reduce w.r.t $A \rightarrow b$

PUSH A, and PUSH Goto[2, A]







Action [S, $] = R_1

i.e. Reduce w.r.t.

$S \rightarrow AA$

→ Pop − 4

→ PUSH S

→ Push Goto [0, S] = 1



Action [1, $] = Accept

∴ Parsing Successful



LR(0) Parser :-

- L → Left to Right scan
- R → Reverse of RMD
- (0) → '0' look-ahead is used to place reduce entry in the Action part of LR(0) Parsing table

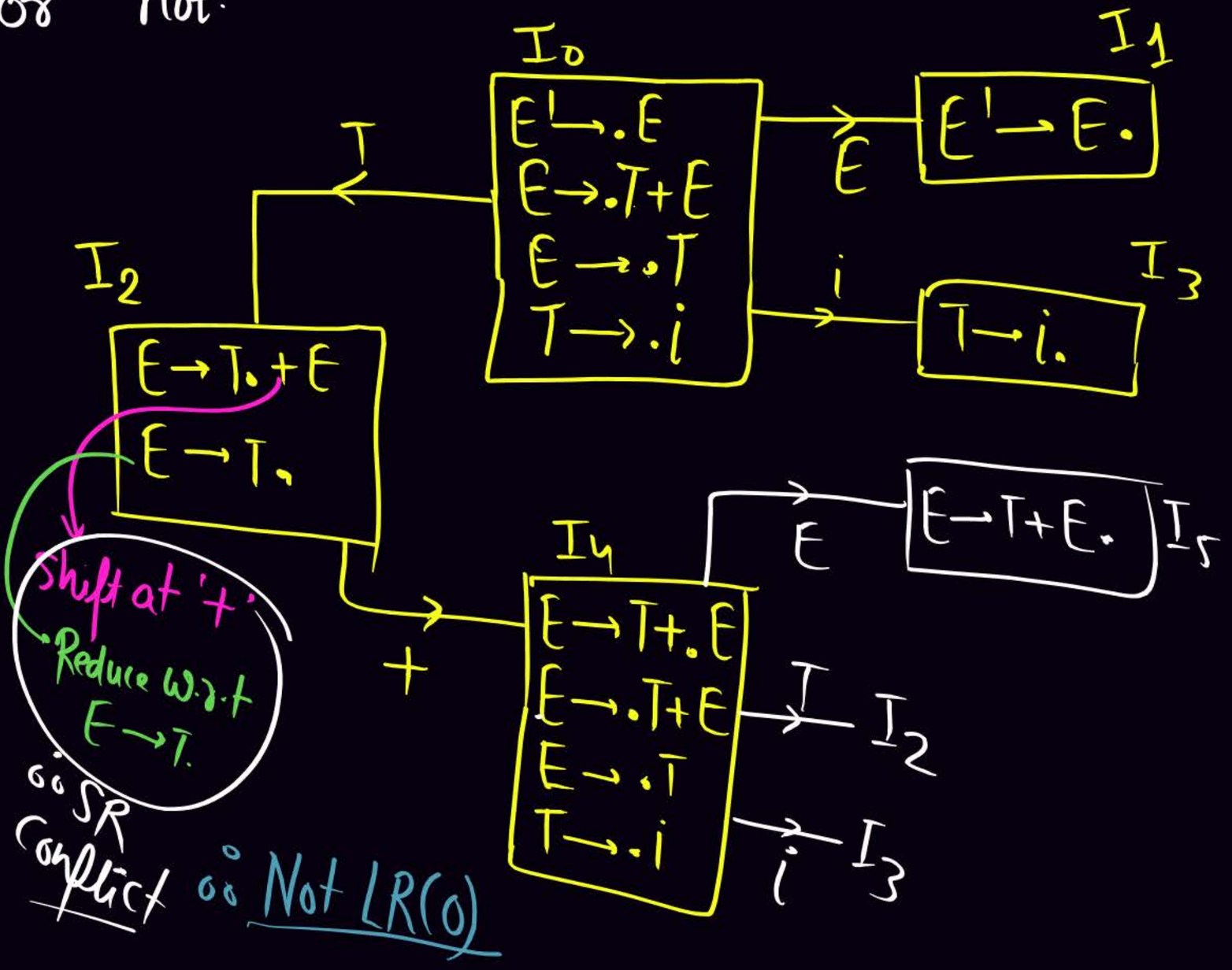
{ Reduce action is set in all the entries of Action part of respective state }

Q:- Check whether the gramman is LR(0) gramman or not.

$$G_1 = \begin{cases} E \longrightarrow T+E \\ E \longrightarrow T \\ T \longrightarrow i \end{cases}$$

Augmented $G_1 =$

$$\begin{cases} E' \longrightarrow E \\ E \longrightarrow T+E \\ E \longrightarrow T \\ T \longrightarrow i \end{cases}$$



∴ SR Conflict ∴ Not LR(0)

Q Check whether LR(0) or not ?

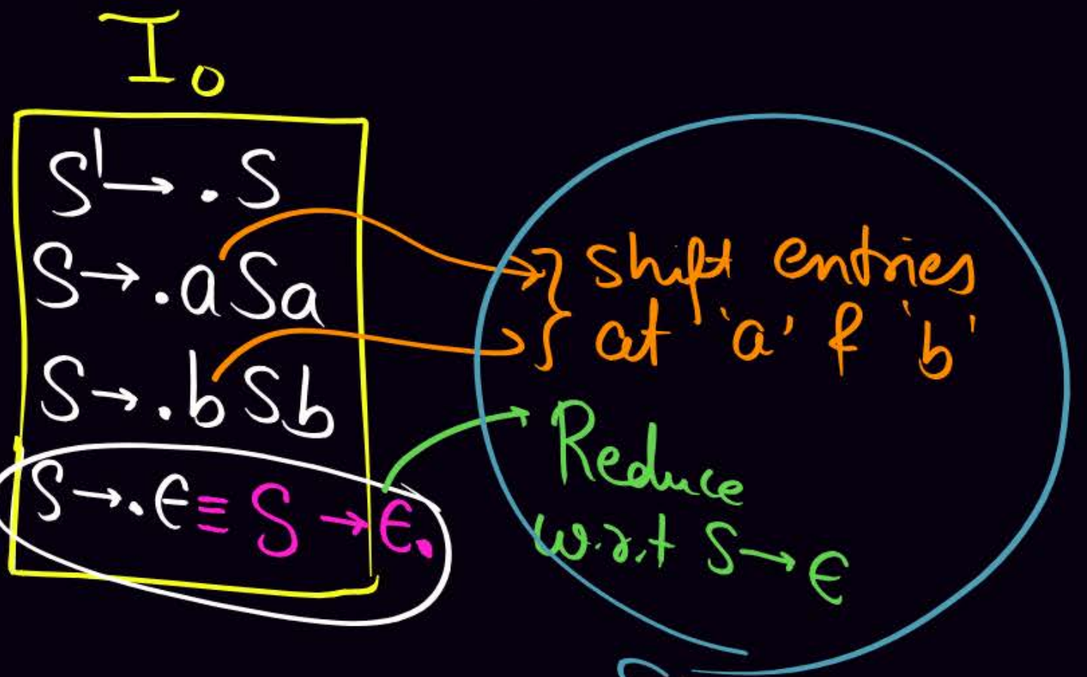
$A \rightarrow .\epsilon \equiv A \rightarrow \epsilon.$

① $S \rightarrow aSa$

② $S \rightarrow bSb$

③ $S \rightarrow \epsilon$

Augmented G: $S' \rightarrow S$, $S \rightarrow aSa$, $S \rightarrow bSb$, $S \rightarrow \epsilon$



SR Conflict
∴ Not LR(0)



2 mins Summary

Topic LR(0) Parser

Topic SLR(1) Parser

Topic CLR(1) & LALR(1) Parser

THANK - YOU